IT550 · INFORMATION RETRIEVAL · COURSE PROJECT

Generate, Retrieve, Decide

HyDE query expansion with Qwen3, and calibrated decision-model re-ranking with Clef, evaluated on TREC-COVID

Team handbook — version 1.0 — 20 October 2026

This document is the single source of truth for the project. It records what we are building, why each choice was made, which external code and data we are reusing, the exact step-by-step procedure for every stage, and how the results are evaluated. Anyone joining the project should be able to read this document and become productive without needing anything explained verbally.

Field	Value
Project title	Generate, Retrieve, Decide: HyDE Query Expansion with Qwen3 and Calibrated Decision-Model Re-ranking on TREC-COVID
Course	IT550 Information Retrieval
Form category	Combination (research paper + TREC track)
Anchor paper	Gao, Ma, Lin & Callan — *Precise Zero-Shot Dense Retrieval without Relevance Labels* (HyDE), ACL 2023 — arxiv.org/abs/2212.10496
Test collection	TREC-COVID (TREC 2020 track), accessed through BEIR
Generator model	Qwen3 instruct, served locally via Ollama (Apache 2.0)
Re-ranker model	Clef / clef-flash, 27B decision model, served locally via Ollama (Apache 2.0)
Hardware	1 × NVIDIA RTX 5090, 32 GB VRAM
Primary metric	nDCG@10
Duration	8 weeks

How to keep this document alive
Every decision that changes the method, the data, or the evaluation goes into the **Decision Log** (Section 16) with a date and a one-line reason. Do not change the body of this document silently — if Section 7 says we use top-100 candidates and we switch to top-50, that is a decision log entry *and* an edit. The log is what makes the final report writable in week 8 without trying to remember February.

Contents

#	Section	What it covers
1	Project summary	What we are building, in one page
2	Background: the two problems	Why a standard pipeline leaves quality unclaimed
3	Stage A — HyDE query expansion	Method, prompt, parameters
4	Stage B — Clef decision re-ranking	Method, API, question types

#	Section	What it covers
5	Design decisions and rejected alternatives	Why each choice, and what we did not pick
6	Prior art and what is actually new	Honest novelty accounting
7	The dataset: TREC-COVID	Statistics, qrels, reference numbers to reproduce
8	External repositories and resources	What we take from each, and what we do not
9	Repository layout and file contracts	Exact schemas for every intermediate file
10	Stage 0 — environment setup	Step-by-step, with acceptance checks
11	Stage 1 to Stage 6 — the build	The full procedure, stage by stage
12	Systems under comparison	S1 to S7, and what each run requires
13	Evaluation protocol	Metrics, significance testing, result templates
14	The calibration study	Our distinctive contribution, in detail
15	Single-GPU operations playbook	Model sequencing on 32 GB VRAM
16	Risks, decision log, and open questions	Living sections
17	Timeline, ownership, and conventions	Who does what, and how we work
18	Viva preparation	Likely questions and our answers
19	Glossary	Terms any team member may need
20	References	Everything cited, with links

1 Project summary

What we are building, in one page

One line
One model imagines the document we are looking for; a second model decides how relevant each retrieved candidate actually is — and we measure what each stage is worth separately.

A search query is three words. A relevant document is a two-hundred-word scientific abstract. They do not look alike, which is why matching them directly is hard. And once candidates are retrieved, a BM25 score or a cosine similarity has never actually *read* the document — it has scored word overlap or vector proximity.

We attack both ends. **Stage A** asks Qwen3 to write a hypothetical abstract that would answer the topic, embeds that instead of the bare query, and retrieves the top 100 candidates. **Stage B** passes each candidate to Clef — a decision model that returns a calibrated probability rather than text — and re-sorts by P(relevant).

Pipeline

```
CORPUS (171,332 docs)                      TOPIC (50 queries)
      |                                           |
      | stage 1: encode once                      | stage 2: generate
      v                                           v
  FAISS index  <------------------------  hypothetical abstract (Qwen3)
      |                                           |
      |                  stage 3: embed + average + search
      v
  TOP-100 CANDIDATES  (frozen to disk — identical for every re-ranker)
      |
      | stage 4: for each candidate, ask Clef "is this relevant?"  ->  P in [0,1]
      v
  RE-RANKED LIST  --->  stage 5: evaluate vs TREC qrels  (nDCG@10, MAP, P@10, Recall@100)
                   --->  stage 6: calibration study (reliability diagram, ECE)
```

What makes this a research project rather than a tutorial

- **Clef has never been evaluated on retrieval.** Its published benchmarks are intent classification and tool calling. There is no IR number for it anywhere. Whatever we measure is the first data point.
- **We test whether its probabilities are calibrated** against TREC's own graded relevance judgments. Almost no re-ranking paper checks this; they report nDCG and stop.
- **We hold the candidate set fixed** across all four re-rankers, so any difference is attributable to the re-ranker alone rather than to first-stage variation.

What we are not claiming
Pointwise LLM re-ranking by probability is not new — monoT5 scores documents by the softmax over 'true' / 'false' tokens, and UPR scores by query-regeneration likelihood. We say so explicitly in Section 6. Overclaiming novelty is the fastest way to lose marks in a viva.

2 Background: the two problems

Why a standard zero-shot pipeline leaves quality unclaimed

Problem 1 — the query side: queries and documents do not look alike

Dense retrieval embeds the query and the document into one vector space and compares them. But the two objects are structurally different. A TREC-COVID topic is a handful of words in interrogative form. A relevant document is a long declarative abstract written in clinical vocabulary. Three specific failure modes follow:

Failure mode	What goes wrong	Example
Vocabulary mismatch	The query's words barely appear in the relevant document, which uses domain terminology instead.	*coronavirus origin* vs. an abstract about *sarbecovirus phylogeny in horseshoe bat populations*
Length asymmetry	A 3–5 token query and a 200-token abstract land far apart in embedding space even when topically identical, because the encoder was trained on different length distributions.	cosine similarity is depressed by length alone
No training labels	TREC-COVID has 50 topics and no training split, so fine-tuning a retriever on in-domain relevance data is not available to us.	zero-shot is forced, not chosen

Problem 2 — the ranking side: first-stage scores are not relevance

Whatever the first stage returns, its scores are a proxy. They are not a judgment about whether the document answers the question.

Signal	What it actually measures	Why that is not relevance
BM25 score	Weighted term overlap, saturating in term frequency and normalised by document length.	An unbounded number with no probabilistic meaning. A score of 18.4 tells you nothing on its own.
Cosine similarity	Angular proximity of two embeddings.	Two vectors being close means the texts are topically similar, not that one answers the other.
Both	Nothing read the document.	No model has formed an opinion about whether this specific abstract addresses this specific information need.

The structure of our answer

HyDE closes the first gap by making the query-side object *look like a document*. Clef closes the second by having a model actually read the pair and return a probability. Because we measure the two stages separately, we can say which one earned the improvement.

3 Stage A — HyDE query expansion

Method, prompt design, and parameters

The method in four steps

1. **Generate.** An instruction-tuned LLM is prompted to write a passage that would answer the query, in the register of the target corpus. The passage may contain factual errors; this is accepted and expected.
2. **Encode.** The generated passage, and the original query, are embedded with a contrastively trained dense encoder (we use Contriever, matching the original paper).
3. **Average.** The N generated-passage embeddings and the query embedding are averaged into a single dense vector.
4. **Retrieve.** That averaged vector drives nearest-neighbour search over the pre-encoded corpus index. No training, no relevance labels, no feedback loop.

Why hallucination is survivable — and the answer to the obvious viva question

The dense encoder acts as a lossy compressor. It preserves what the generated passage is *about* — its topic, its register, its domain vocabulary — and discards most of the specific invented detail, because that detail is not what the embedding dimension budget is spent on. Averaging over N independent generations further cancels idiosyncratic fabrications while reinforcing the signal common to all of them. And in our pipeline there is a second safeguard the original paper does not have: Stage B re-reads every candidate against the *real* document text, so a hallucination that survives into retrieval still has to pass a relevance judgment.

Why HyDE rather than classical pseudo-relevance feedback

Classical PRF (Rocchio, RM3) runs the query, assumes the top-k results are relevant, harvests terms from them, and re-runs. The weakness is structural: **if the first retrieval is poor, you expand the query with terms from irrelevant documents and make it worse.** HyDE never touches the corpus during expansion. The LLM writes from parametric knowledge, so expansion quality is independent of retrieval quality. On a collection like TREC-COVID, where topics are short and clinical, that independence matters.

Prompt specification

The prompt is **frozen after week 3** and recorded here. Changing it invalidates every downstream run, so a change is a decision-log entry and a full regeneration.

```
PROMPT_V1 = """Write a short scientific abstract that answers the following question.
Write it in the style of a COVID-19 research paper abstract: formal, specific,
and using domain terminology. Do not refuse, do not add disclaimers, and do not
mention that this is hypothetical. Write 80-120 words.

Question: {query}

Abstract:"""
```

Parameter	Value	Reason
Model	qwen3:8b (primary), qwen3:4b (ablation)	Apache 2.0, runs locally, no API cost
N generations	8	The HyDE paper uses 8; averaging needs enough samples to cancel noise
temperature	0.7	Enough diversity across the 8 samples to make averaging meaningful

Parameter	Value	Reason
seed	recorded per generation	Reproducibility — see the file contract in Section 9
max tokens	200	80–120 words plus headroom
Query inclusion	query embedding averaged in alongside the 8	Matches the paper; keeps the original intent anchored if generations drift

Thinking mode must be disabled
Qwen3 is a hybrid reasoning model. If thinking mode is active, the output will contain reasoning traces that must not be embedded as a hypothetical document. Disable it in the Ollama request (or strip everything before the final answer) and verify by inspecting the first five generations by hand before running all 50 topics.

4 Stage B — Clef decision re-ranking

What Clef is, how it is called, and how we turn it into a ranker

What Clef is

Clef is a 27-billion-parameter **decision model** published by Cloudflare and distributed through Ollama under the Apache 2.0 licence. Its model card states it is fine-tuned from a Qwen3-family 27B base. Critically, **it is not a text generator.** You supply a *state* (text, structured JSON, or images) together with between 1 and 64 typed questions, and it returns probabilities. That output shape is, almost exactly, the shape of a relevance judgment — which is what makes it usable as a re-ranker at all.

Property	Value	Source / note
Parameters	27B	Model card
Tags	clef:latest, clef:27b (≈18–20 GB); clef-flash (smaller, faster)	Model card
Licence	Apache 2.0	Model card
Endpoint	POST /v1/systemone	e.g. http://localhost:11434/v1/systemone
Minimum runtime	Ollama 0.35.1 or later	Model card
Median latency	209.3 ms (clef) / 38.8 ms (clef-flash)	Cloudflare's own evaluation
p95 latency	238.6 ms (clef) / 122.4 ms (clef-flash)	Cloudflare's own evaluation
Context window	64K per the highlights; 256K per the model tags	**Inconsistent on the model card — verify empirically in Stage 4 before trusting long states**
SDK support	Not supported in the Ollama CLI or official Python / JS libraries	We call the HTTP endpoint directly

The three question types, and which we use

Type	Returns	Our use
noul	`noul` — the probability that a yes/no statement is true	**Primary.** We ask whether the document is relevant and use the returned probability directly as the ranking score. This is system S5.
score	`score` (probability-weighted level), `legend`, `probabilities`, `confidence`	**Secondary.** Configured with three levels to mirror TREC-COVID's graded 0 / 1 / 2 judgments. This is system S6 and is what makes the graded qrels directly exploitable.
choice	`choice`, `probabilities`, `confidence` over 2–26 options	Not used. Retained as a fallback framing only if noul and score both behave badly.

Request shape

```
POST http://localhost:11434/v1/systemone

{
  "model": "clef",
  "state": "TOPIC: <query text>\n\nQUESTION: <topic question field>\n\n"
           "DOCUMENT TITLE: <title>\n\nDOCUMENT: <abstract text>",
  "questions": [
    {
      "name": "relevant",
      "type": "noul",
      "prompt": "The DOCUMENT is relevant to the TOPIC and helps answer the QUESTION."
    }
  ],
  "keep_alive": 0
}

-> response contains, per question name, the probability in [0, 1]
```

keep_alive is the VRAM lever
Setting `keep_alive: 0` tells Ollama to unload the model as soon as the request completes. During a batch run we leave it at the default so the model stays resident between the 5,000 calls; we set it to 0 on the *final* call of a stage so the GPU is clear before the next stage loads a different model. See Section 15.

Turning probabilities into a ranking

For each of the 50 topics we hold a fixed list of 100 candidates from Stage A. For each (topic, candidate) pair we issue one request and read the probability. The ranking is then simply a descending sort of those 100 probabilities. Ties are broken by the original Stage A rank, so the re-ranker can never do worse than random on a tie. Every raw response is written to disk before sorting (Section 9), because the probabilities themselves — not just the ordering — are the input to the calibration study in Section 14.

5 Design decisions and rejected alternatives

Every significant choice, with what we did not pick and why

This section exists so that nobody re-litigates a settled question in week 6, and so that the viva answer to “why did you do it that way?” is already written down.

ID	Decision	Chosen	Rejected	Reason
D1	Query expansion method	**HyDE**	Rocchio / RM3 feedback; Query2doc; no expansion	PRF depends on first-stage quality — bad retrieval produces bad expansion terms. HyDE is independent of the corpus. Query2doc is close kin but appends to the query lexically; HyDE operates in embedding space, which is the half of the pipeline we want to study.
D2	Re-ranker	**Clef (decision model)**	RankGPT-style listwise LLM; monoT5; cross-encoder only	A listwise LLM emits a permutation as text that must be parsed, and it skips or duplicates IDs. Clef returns a bounded number with a confidence. The output is the right *type* for ranking, and it enables the calibration study that no alternative supports.
D3	Test collection	**TREC-COVID**	Touché 2020; FIRE AILA; TREC DL 2019	Graded 0/1/2 judgments (required to test the `score` question type), 50 topics, one-line load through BEIR, no licence friction, and published reference numbers we can reproduce as a correctness check.
D4	Generator	**Qwen3 instruct, local**	GPT-3 / GPT-4 via API (as the original paper uses)	Apache 2.0, zero cost, no API key, fully reproducible by a marker. Also makes the generator-size ablation free.
D5	First-stage encoder	**Contriever**	E5; GTE; BGE	Matches the HyDE paper, so our Stage A is a faithful reproduction before we extend it. A second encoder is kept as a robustness check, not as the headline.
D6	Pipeline architecture	**Staged, files on disk**	Single in-process loop over queries	Forced by 32 GB VRAM (one model resident at a time), but correct regardless: it freezes the candidate set so re-ranker comparisons are unconfounded, makes any stage re-runnable in isolation, and makes a crash cost one stage.
D7	Re-ranking depth	**Top-100**	Top-50; top-1000	100 × 50 topics = 5,000 decisions, under an hour at the published latency. Top-1000 would be 50,000 calls for little gain, since nDCG@10 is the primary metric.
D8	Supervised baseline	**MiniLM cross-encoder**	No re-ranking baseline at all	Without a trained re-ranker in the table, “Clef beats BM25” is near-meaningless. MiniLM (cross-encoder/ms-marco-MiniLM-L-6-v2) is the honest bar: supervised, trained on ranking data, widely reported.
D9	Primary metric	**nDCG@10**	MAP; MRR; P@10	The BEIR and TREC-COVID standard, uses the graded judgments, and focuses on the top of the ranking where re-ranking acts.

6 Prior art and what is actually new

Honest novelty accounting

Read this before the viva
An examiner will ask what is new. The worst possible answer is to claim the pipeline is novel — two of its three ideas are published. Lead with items N1 and N2 below, concede the rest cheerfully, and the question becomes a strength rather than an ambush.

Established work we are standing on (not novel)

Work	What it established	Relation to us
HyDE — Gao et al., ACL 2023	LLM-generated hypothetical documents improve zero-shot dense retrieval.	We reproduce it, with a local open-weight generator and on a different collection. Our Stage A is **their method**.
monoT5 — Nogueira et al., 2020	Pointwise LLM re-ranking by reading the softmax over `true` / `false` tokens.	**This is the closest prior art to Stage B.** Scoring documents by a model's probability is not our idea. Say so first.
UPR — Sachan et al., 2022	Re-ranking by the likelihood that the model regenerates the query given the document.	Another probability-based re-ranker. Same family.
RankGPT — Sun et al., EMNLP 2023	Listwise LLM re-ranking by prompting for a permutation.	The alternative we rejected (D2). We contrast against its output-shape weakness.
LLM relevance judges — Thomas et al., SIGIR 2024; UMBRELA	LLMs can predict human relevance labels well enough to rank systems consistently.	Related but a different task: they replace annotators, we rank documents.
Cross-encoders — Nogueira & Cho, 2019	Supervised re-ranking of first-stage candidates.	Our baseline, system S4.

What is genuinely new in this project

ID	Contribution	Why it counts
N1	First IR evaluation of Clef	Clef's published benchmarks are BANKING77, CLINC150, BFCL, API-Bank, RouterBench and similar — intent classification, tool calling and routing. **There is no published retrieval number for this model.** Whatever we measure is the first data point, and that is true whether the result is positive or negative.
N2	Calibration of a re-ranker's scores against graded qrels	Re-ranking papers report nDCG and stop. We bin Clef's returned probabilities and plot the observed relevance rate per bin against TREC's own judgments, with expected calibration error. If P = 0.8 really corresponds to ~80% judged relevant, a *single threshold transfers across topics* — a capability no score-based re-ranker offers. This is our strongest and most distinctive figure.
N3	Purpose-built decision model vs. coaxed token probabilities	monoT5's probability is a by-product: you read logits from a model trained to produce text. Clef was *built* to emit calibrated probabilities as its primary interface. Whether that architectural difference yields better-calibrated relevance scores is an open empirical question, and we answer it.
N4	Attributing the gain to a stage	Most pipeline papers report one end-to-end number. We freeze the candidate set and vary only the re-ranker across S4–S7, so the improvement from expansion and the improvement from re-ranking are separately measurable.

Known weaknesses, and how we answer them

Challenge	Our answer
“Clef just loses to the cross-encoder.”	Likely, and acceptable. A model trained on ranking data beating one that never saw the task is the expected outcome. The contribution is the measurement plus the calibration finding, not a win. We state this in the hypotheses before running anything.
“You assembled two existing components.”	True of the architecture. The contribution is N1 and N2 — evaluating an unevaluated model and testing whether its probabilities mean what they claim.
“50 topics is a small sample.”	Which is exactly why every comparison is reported with a paired significance test rather than a raw delta.
“The generator hallucinates.”	The encoder is lossy, we average 8 generations, and Stage B re-reads the real document. See Section 3.

7 The dataset: TREC-COVID

Statistics, judgments, and the reference numbers we must reproduce

TREC-COVID was a TREC track run in 2020 over CORD-19, the COVID-19 Open Research Dataset. Topics were written by biomedical experts and judged by people with medical training. We access it through BEIR, which redistributes a standardised version with the qrels bundled.

Property	Value	Consequence for us
Documents	171,332	Encoding takes roughly 20–40 minutes on the RTX 5090. Do it once, cache to disk, never repeat.
Test queries (topics)	50	Small. Every comparison needs a paired significance test.
Avg. relevant docs per query	493.5	Unusually high. See the note below — it changes how we read Recall.
Relevance scale	Graded: 0 = not relevant, 1 = partially relevant, 2 = fully relevant	This is what makes Clef's `score` question type (S6) testable and what the calibration study is validated against.
Topic fields	query, question, narrative	We use *query* and *question*. The narrative is held out — using it is a documented variant, not the default.
Document fields	title, abstract (text)	State construction for Clef uses title + text.
Licence / access	Public, via BEIR	No registration, no gatekeeping.

493.5 relevant documents per topic changes how we read Recall

With that many relevant documents, Recall@100 is bounded very low by construction — retrieving 100 documents cannot recover 493. **Do not report low Recall@100 as a failure.** It is a property of the collection. Recall@100 is in our table as a *ceiling check*: it tells us how much relevant material Stage A handed to the re-ranker, and therefore bounds what any re-ranker could possibly achieve. nDCG@10 is the metric that answers the research questions.

Published reference numbers — our correctness check

These are from the BEIR paper (Thakur et al., NeurIPS 2021 Datasets & Benchmarks). Reproducing the BM25 row is **the acceptance test for Stage 1 and Stage 3**. If our BM25 does not land near 0.656, something is wrong with the index, the qrels alignment, or the evaluation call — and we fix that before generating a single hypothetical document.

System	nDCG@10 on TREC-COVID	Note
BM25	0.656	**Our Stage 1 acceptance target.** Note how strong it is — TREC-COVID is a collection where lexical matching does well.
ANCE	0.654	A dense retriever roughly level with BM25 here.
TAS-B	0.481	A dense retriever clearly below BM25 — dense models are not automatically better on this collection.
BM25 + cross-encoder	0.757	**The number our re-rankers are chasing.** Re-ranking is worth roughly +0.10 nDCG@10 here when done with a trained model.

What these numbers tell us before we start

Two things. First, the headroom from re-ranking on this collection is real and large (0.656 → 0.757), so Stage B has room to show an effect. Second, dense retrieval is not a free win here, which makes Stage A's contribution a genuine question rather than a formality.

8 External repositories and resources

What we take from each — and, equally important, what we do not

Primary reference implementation

Field	Detail
Repository	`github.com/texttron/hyde`
Paper	Gao, Ma, Lin & Callan — *Precise Zero-Shot Dense Retrieval without Relevance Labels*, ACL 2023 — `arxiv.org/abs/2212.10496`
Contents	`src/hyde/` (package source), `hyde-demo.ipynb` (single-query walkthrough), `hyde-dl19.ipynb` (TREC DL19 experiment), `setup.py`, `approach.png`
Their stack	pyserini for dense retrieval and evaluation; a prebuilt Contriever FAISS index downloaded from Dropbox; GPT-3 via the OpenAI API as generator (`export OPENAI=<key>`)

What we take from it

- **The method and its parameters** — the generate / encode / average / retrieve structure, and the choice of 8 generations averaged with the query embedding.
- **The prompt pattern** — a short instruction that forces the model to write a passage in the target corpus's register, adapted by us to scientific-abstract style.
- **The architectural decomposition** — promptor, generator, encoder, searcher as separate concerns. Our staged scripts mirror this cleanly.
- **Its results as a sanity reference** — if our Stage A shows no improvement at all over query-only dense retrieval, that contradicts a replicated published result and signals a bug in our code, not a finding.

What we deliberately do not take

Their choice	Our choice	Why
GPT-3 via the OpenAI API	Qwen3 served locally by Ollama	Cost, reproducibility by a marker, and no API key in the repo. Also makes the generator-size ablation free.
Prebuilt Contriever index over MS MARCO, from Dropbox	Our own FAISS index over TREC-COVID	**Their index is built over MS MARCO, for the TREC DL19 experiment. It does not contain our documents.** We must build our own in Stage 1 — this is the single most common misunderstanding when someone first reads their README.
pyserini for search and evaluation	faiss-cpu/gpu + ranx	pyserini pulls in a Java toolchain. Our pipeline is pure Python, which is one less thing to install on four machines. (pyserini remains a fallback if we want its BM25.)
Jupyter notebooks as the experiment driver	Numbered CLI scripts writing files	Notebooks hide execution order and hold state invisibly. A staged script pipeline is re-runnable, diffable, and survives the single-GPU constraint.

Everything else we depend on

Resource	Link	What it gives us
BEIR	`github.com/beir-cellar/beir`	One-line download of TREC-COVID with corpus, queries and qrels; `GenericDataLoader` and `EvaluateRetrieval`
ir_datasets	`ir-datasets.com`	Alternative loader; useful for cross-checking that our qrels parse matches a second implementation

Resource	Link	What it gives us
ranx	`github.com/AmenRa/ranx`	`Qrels`, `Run`, `evaluate()`, `compare()` (paired Student's t-test), `fuse()` for RRF
sentence-transformers	`sbert.net`	Contriever encoding, and the `CrossEncoder` class for the S4 baseline
Contriever	`facebook/contriever` on Hugging Face	The unsupervised dense encoder used by the HyDE paper
Cross-encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`	The supervised re-ranking baseline (S4)
FAISS	`github.com/facebookresearch/faiss`	Vector index. Flat inner product — exact search, since 50 queries makes ANN speed irrelevant
Qwen3	`github.com/QwenLM/Qwen3` · `ollama.com/library/qwen3`	The generator, Apache 2.0
Clef	`ollama.com/library/clef`	The decision model, Apache 2.0
Ollama	`ollama.com`	Local model serving; version 0.35.1+ required for decision models

9 Repository layout and file contracts

The schemas that let four people work without colliding

Every stage reads files and writes files. The contracts below are binding: if you change a schema, every downstream stage breaks, so it is a decision-log entry and an announcement to the team.

```
hyde-clef-trec-covid/
├── README.md                 # 10-line quickstart; points here for everything else
├── requirements.txt          # pinned versions
├── config.yaml               # ALL tunables; no magic numbers in scripts
├── data/
│   ├── raw/trec-covid/       # BEIR download (corpus.jsonl, queries.jsonl, qrels/)
│   └── cache/
│       ├── corpus_emb.npy    # (171332, 768) float32
│       ├── docids.json       # row index -> doc id   [CRITICAL: see below]
│       └── faiss.index
├── scripts/
│   ├── 01_download.py
│   ├── 02_index.py           # GPU: Contriever
│   ├── 03_generate.py        # GPU: Qwen3
│   ├── 04_retrieve.py        # GPU: Contriever
│   ├── 05_rerank_ce.py       # GPU: MiniLM cross-encoder   (S4)
│   ├── 06_rerank_clef.py     # GPU: Clef                   (S5, S6, S7)
│   ├── 07_evaluate.py        # CPU
│   └── 08_calibration.py     # CPU
├── runs/                     # TREC-format run files, one per system
│   ├── S1_bm25.trec
│   ├── S3_hyde.trec
│   └── S5_clef_noul.trec ...
├── artifacts/
│   ├── hyde_docs.jsonl       # generations      (contract A)
│   └── clef_raw.jsonl        # raw decisions    (contract B)
├── results/
│   ├── metrics.csv
│   ├── significance.txt
│   └── calibration.png
└── notebooks/                # exploration ONLY; never the source of a reported number
```

The docids.json mapping is the highest-risk artefact in the project
FAISS returns *row indices*, not document IDs. If the order of `corpus_emb.npy` and `docids.json` ever diverges, every run file silently points at the wrong documents and every metric becomes meaningless — **with no error raised.** Write both in the same loop, in the same script, and never regenerate one without the other. The Stage 1 acceptance check exists primarily to catch this.

Contract A — `artifacts/hyde_docs.jsonl`

One line per generation. 50 topics × 8 generations = 400 lines. **Append-only. Never overwrite.** These generations are the experiment; if they change silently between runs, results stop being reproducible and nobody notices until numbers move for no reason.

```
{"qid": "1",
 "gen_idx": 0,
 "model": "qwen3:8b",
 "prompt_version": "v1",
 "temperature": 0.7,
 "seed": 12345,
 "text": "Phylogenetic analyses of sarbecovirus genomes sampled from ...",
 "n_tokens": 118,
 "created_at": "2026-10-28T14:03:11Z"}
```

Contract B — `artifacts/clef_raw.jsonl`

One line per (topic, candidate) decision. 50 × 100 = 5,000 lines per system. The raw probability is kept because the **calibration study needs the probabilities, not just the ordering** — if you only save

the sorted run file, Section 14 becomes impossible.

```
{"qid": "1",
 "docid": "ug7v899j",
 "model": "clef",
 "question_type": "noul",
 "probability": 0.8137,
 "confidence": 0.91,
 "stage_a_rank": 7,
 "latency_ms": 212,
 "state_chars": 1840,
 "truncated": false}
```

Contract C — `runs/*.trec` (standard TREC run format)

```
qid  Q0  docid      rank  score      tag
1    Q0  ug7v899j   1     0.813700   S5_clef_noul
1    Q0  02tnwd4m   2     0.794100   S5_clef_noul

Six whitespace-separated columns. rank is 1-based. score must decrease
monotonically with rank. tag matches the system ID exactly.
```

10 Stage 0 — environment setup

Do this once per machine, then run the acceptance check

Step 0.1 — Python environment

```
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install --upgrade pip

pip install beir ir_datasets ranx sentence-transformers transformers torch
pip install faiss-cpu numpy pandas matplotlib scikit-learn scipy requests pyyaml tqdm
# If a CUDA build of faiss is available for your platform, prefer faiss-gpu.
# faiss-cpu is fine here: 171k vectors with flat search is fast enough.
```

Step 0.2 — Ollama and the models

```
# Install Ollama (see ollama.com). Decision models need 0.35.1 or later.
ollama --version          # MUST be >= 0.35.1, otherwise /v1/systemone will not exist

ollama pull qwen3:8b      #  ~5-6 GB
ollama pull qwen3:4b      #  ~3 GB   (generator-size ablation)
ollama pull clef          # ~18-20 GB
ollama pull clef-flash    #  smaller/faster variant

ollama list               # confirm all four are present
```

Step 0.3 — Verify the Clef endpoint before anything else

This is the one piece of the stack with no SDK support, so prove it works on day one. Discovering in week 5 that the endpoint is unavailable would be expensive.

```
curl http://localhost:11434/v1/systemone -d '{
  "model": "clef",
  "state": "DOCUMENT: Aspirin reduces fever. TOPIC: fever treatment",
  "questions": [{"name":"relevant","type":"noul",
                 "prompt":"The DOCUMENT is relevant to the TOPIC."}]
}'

# Expect a JSON body containing a probability for "relevant".
# A 404 means your Ollama is too old. A connection refused means it is not running.
```

Stage 0 acceptance check

You are done when: (a) `ollama --version` reports 0.35.1 or later; (b) all four models appear in `ollama list`; (c) the curl above returns a probability between 0 and 1; and (d) you have written down the actual usable context length you observe, because the model card is self-contradictory on this (64K vs 256K). Record the answer in the decision log.

Step 0.4 — config.yaml

Every tunable lives here. No magic numbers inside scripts — when a parameter appears in two places, the two places eventually disagree.

```
dataset:       {name: trec-covid, split: test}
encoder:       {model: facebook/contriever, batch_size: 256, max_length: 350}
generator:     {model: qwen3:8b, n_generations: 8, temperature: 0.7,
                max_tokens: 200, prompt_version: v1, thinking: false}
retrieval:     {top_k: 100, include_query_embedding: true}
reranker:      {clef_model: clef, question_type: noul, keep_alive: "5m",
                max_state_chars: 6000}
cross_encoder: {model: cross-encoder/ms-marco-MiniLM-L-6-v2}
eval:          {metrics: [ndcg@10, map, precision@10, recall@100], max_p: 0.05}
seed: 42
```

11 Stage 1 to Stage 6 — the build

The full procedure. Each stage: what, how, output, acceptance check.

The golden rule of this pipeline
One model on the GPU at a time. Each stage loads what it needs, writes its output to disk, and exits. No stage ever holds two models. This is what makes the project run on a single 32 GB card, and it is also what makes every stage independently re-runnable.

Stage 1 — Download and index the corpus

Field	Detail
Script	`01_download.py`, `02_index.py`
Model on GPU	Contriever only (~1 GB)
Runtime	Download a few minutes; encoding roughly 20–40 minutes on the RTX 5090
Outputs	`data/cache/corpus_emb.npy`, `docids.json`, `faiss.index`

Procedure

1. Download TREC-COVID through BEIR's `util.download_and_unzip` and load it with `GenericDataLoader(...).load(split='test')`. You get `corpus`, `queries`, `qrels`.
2. Assert the shapes immediately: 171,332 documents and 50 queries. If these differ, stop — you have the wrong split or a partial download.
3. Build each document's text as `title + ' ' + text`, truncated to the encoder's max length.
4. Encode in batches with Contriever (mean pooling). **Write the doc id to** `docids.json` **inside the same loop that writes the embedding row.**
5. Build a FAISS flat inner-product index (`IndexFlatIP`) over the normalised embeddings and persist it.
6. Build a BM25 index over the same texts for system S1, and produce `runs/S1_bm25.trec` straight away.

Stage 1 acceptance check — do not proceed until this passes
Evaluate `runs/S1_bm25.trec` against the qrels. **nDCG@10 must land near 0.656** (the published BEIR figure). A value far below suggests broken qrels alignment or a docid mapping error; a value far above suggests you are evaluating on something other than the test split. This single number validates the loader, the docid mapping, and the evaluation call all at once — which is why it comes before any LLM work.

Stage 2 — Generate hypothetical documents (Qwen3)

Field	Detail
Script	`03_generate.py`
Model on GPU	Qwen3 only (~5–6 GB for 8b). Contriever is **not** loaded in this stage.
Work	50 topics × 8 generations = 400 short passages
Runtime	A few minutes on the RTX 5090
Output	`artifacts/hyde_docs.jsonl` (contract A)

Procedure

1. Load the 50 topics. Use the *query* field as the prompt input; record the *question* field too, since Stage 4 uses it in the state.

2. For each topic, call Ollama 8 times with `PROMPT_V1`, `temperature 0.7`, and an explicit recorded seed per call.
3. **Disable thinking mode** and verify: print the first five generations and read them. They must be plain abstracts with no reasoning trace, no refusal, no "As an AI" preamble, no markdown headers.
4. Append each generation to the JSONL with its full metadata. Never overwrite the file; if you must regenerate, move the old file to `hyde_docs.v1.jsonl` and record why in the decision log.
5. Spot-check length: generations should be roughly 80–120 words. Systematically short output usually means the prompt is being truncated or thinking mode is eating the budget.

Stage 2 acceptance check
400 lines in the JSONL; every line has a non-empty `text`; a manual read of five random generations shows plausible, abstract-style prose on the right topic. **Read them yourself — this is the step where a silent prompt failure can otherwise survive all the way to the results table.**

Stage 3 — Embed, average, and retrieve

Field	Detail
Script	`04_retrieve.py`
Model on GPU	Contriever only. Qwen3 is unloaded — we only read its output file.
Runtime	Under a minute (50 queries against a flat index)
Outputs	`runs/S2_dense.trec`, `runs/S3_hyde.trec`, and the frozen top-100 candidate file

Procedure

1. For S2 (query-only dense): embed each raw topic, search the index, take the top 100.
2. For S3 (HyDE): embed the 8 generations for a topic plus the topic itself, average the 9 vectors, normalise, and search. Taking the mean *after* normalising each component is the convention from the paper — do it consistently and record which you did.
3. Write both run files in TREC format (contract C).
4. **Freeze the S3 top-100 candidate list to its own file.** Every re-ranker in Stage 4 reads *this exact file*. This is what makes S4–S7 a controlled comparison.

Stage 3 acceptance check
S3 should improve over S2 on nDCG@10. If HyDE shows *no* improvement at all over query-only dense retrieval, that contradicts a replicated published result — treat it as a bug (most likely in averaging, normalisation, or the generations themselves) rather than as a finding. Note separately that S3 need not beat BM25's 0.656; on this collection BM25 is strong, and that comparison is part of what we are measuring.

Stage 4 — Re-rank the frozen candidates

Four re-rankers read the *same* 100 candidates per topic and produce four run files. Run them in separate invocations so only one model is resident at a time.

Run	System	Script	Model on GPU	Decisions
4a	S4 — cross-encoder baseline	`05_rerank_ce.py`	MiniLM (~100 MB)	5,000 pair scores
4b	S5 — Clef, `noul`	`06_rerank_clef.py`	clef (~18–20 GB)	5,000 calls
4c	S6 — Clef, `score`	`06_rerank_clef.py`	clef (~18–20 GB)	5,000 calls
4d	S7 — clef-flash	`06_rerank_clef.py`	clef-flash	5,000 calls

State construction — get this right and keep it identical across S5, S6 and S7

```
state = (
    f"TOPIC: {query}\n\n"
    f"QUESTION: {question}\n\n"
    f"DOCUMENT TITLE: {title}\n\n"
    f"DOCUMENT: {abstract}"
)
# Truncate `abstract` to config.reranker.max_state_chars and record
# truncated=true in clef_raw.jsonl when you do. Never truncate silently:
# a systematic truncation bias would be invisible in the metrics but would
# quietly change what the model is judging.
```

Procedure

1. Load the frozen candidate file and the corpus text.
2. For each (topic, candidate), build the state, issue one request, and append the full response to `clef_raw.jsonl` (contract B) — probability, confidence, latency, and whether the state was truncated.
3. **Checkpoint as you go.** Flush the JSONL every 100 decisions so a crash at decision 4,800 does not cost the whole run. On restart, skip pairs already present.
4. After all decisions, sort each topic's candidates by probability descending, breaking ties by the Stage A rank, and write the run file.
5. For S6, configure the `score` question with three levels mapped to the TREC scale (0 not relevant / 1 partially / 2 fully) and rank by the probability-weighted level.
6. Record wall-clock time and mean latency per system — these are results, not diagnostics. RQ4 is an accuracy/latency question.

Stage 4 acceptance check
5,000 lines per system; probabilities span a sensible range (if every probability is 0.5, or every one is above 0.99, the prompt wording is wrong — fix it before reading any metric); and the output ordering actually differs from the Stage A ordering. A re-ranker that changes nothing is a broken re-ranker, not a null result.

Stage 5 — Evaluate

Field	Detail
Script	`07_evaluate.py`
Model on GPU	None — CPU only
Outputs	`results/metrics.csv`, `results/significance.txt`

```
from ranx import Qrels, Run, evaluate, compare

qrels = Qrels(qrels_dict)                      # from the BEIR loader
runs  = [Run.from_file(f"runs/{s}.trec") for s in SYSTEMS]

scores = evaluate(qrels, run, ["ndcg@10", "map", "precision@10", "recall@100"])

report = compare(qrels=qrels, runs=runs,
                 metrics=["ndcg@10", "map", "precision@10"],
                 max_p=0.05)              # two-sided paired Student's t-test
print(report)
```

Stage 5 acceptance check
Every one of the seven systems appears in `metrics.csv`; S1's nDCG@10 still matches the Stage 1 figure (if it moved, something upstream changed and you need to know why); and the significance report is saved, not just printed to a terminal that will be closed.

Stage 6 — Calibration study

Covered in full in Section 14, because it is the distinctive contribution rather than routine evaluation. Script: `08_calibration.py`, CPU only, reading `clef_raw.jsonl` and the qrels.

12 Systems under comparison

Seven runs, one codebase

ID	System	First stage	Re-ranker	Answers	Stage
S1	BM25	BM25	—	Reference	1
S2	Dense, query only	Contriever on raw topic	—	Reference	3
S3	HyDE dense	Contriever on averaged HyDE vector	—	RQ1	3
S4	HyDE + cross-encoder	S3 top-100 (frozen)	MiniLM (supervised)	RQ2	4a
S5	HyDE + Clef (noul)	S3 top-100 (frozen)	Clef, relevance probability	RQ2, RQ3	4b
S6	HyDE + Clef (score)	S3 top-100 (frozen)	Clef, graded 0/1/2 level	RQ3	4c
S7	HyDE + clef-flash	S3 top-100 (frozen)	clef-flash	RQ4	4d

Research questions these answer

RQ	Question	Decided by
RQ1	Does HyDE expansion with Qwen3 improve first-stage retrieval over BM25 and over query-only dense retrieval?	S1 vs S2 vs S3
RQ2	Does Clef re-ranking improve over the first stage, and how does it compare with a trained cross-encoder?	S3 vs S4 vs S5
RQ3	Are Clef's probabilities calibrated against the official graded qrels, and does `score` beat `noul`?	S5 vs S6, plus Section 14
RQ4	What is the accuracy–latency trade-off between clef and clef-flash?	S5 vs S7, with recorded latencies

If the schedule slips, cut in this order

S7 first (RQ4 is the least central), then S6. **Never cut S4** — without the supervised baseline the whole comparison loses its reference point, and “Clef beats BM25” is not a publishable sentence.

13 Evaluation protocol

Metrics, significance, and the tables we will fill

Metric	Role	Why it is here
nDCG@10	Primary	The BEIR and TREC-COVID standard. Uses the graded judgments and concentrates on the top of the ranking, which is where re-ranking acts.
MAP	Secondary	Rewards pushing all relevant documents up rather than only the first page. Catches a system that fixes the top 3 and ruins the rest.
P@10	User-facing	What a reader actually sees on the first screen. Easy to explain in a presentation.
Recall@100	Ceiling check	Fixed by Stage A; bounds what any re-ranker can achieve. Remember the 493.5 relevant-per-topic caveat from Section 7 — read it as a ceiling, not as a score.
Mean latency / query	Cost	RQ4 is explicitly an accuracy-versus-cost question, so latency is a reported result, not a footnote.

Statistical significance

With 50 topics, an unlabelled difference in a table means very little. Every pairwise comparison we report in the final document carries a significance marker.

- **Test:** two-sided paired Student's t-test over per-topic nDCG@10, which is what `ranx.compare()` applies by default.
- **Pairing:** by topic — the same 50 topics run through both systems.
- **Threshold:** $p < 0.05$, with correction for multiple comparisons when the whole 7-system table is compared at once.
- **Reporting:** state the test, the p-value and the direction. “S5 > S3 (+0.04 nDCG@10, $p = 0.03$)”, never a bare bolded number.

Per-query win / loss analysis

For every topic, record whether each re-ranker helped or hurt relative to S3, and by how much. Then pick the two or three topics with the largest losses and explain them: read the generated hypothetical document, read the documents that were promoted, and say what went wrong. **This is where the analysis marks are.** A results table anybody can produce; an explanation of why topic 23 broke is what distinguishes the project.

Main results table — to be filled in week 6

Leave this empty until the runs exist. **No placeholder numbers anywhere, at any point.** A proposal or draft that already “knows” its results is the fastest way to lose an examiner's trust.

System	nDCG@10	MAP	P@10	R@100	ms/query	vs S3
S1 BM25					—	
S2 Dense (query only)					—	
S3 HyDE dense					—	—
S4 + cross-encoder						
S5 + Clef (noul)						
S6 + Clef (score)						
S7 + clef-flash						

Reference points to include in the final write-up: BM25 = 0.656 and BM25 + cross-encoder = 0.757 nDCG@10 as published in the BEIR paper. Our own numbers should be reported alongside these, with any gap discussed rather than hidden.

14 The calibration study

Our distinctive contribution, in full detail

Why this section matters more than the results table

Any team can produce an nDCG table. Almost no re-ranking paper asks whether the scores *mean* anything. Because Clef returns a probability rather than an arbitrary score, we can check it against TREC's own judgments — and that check is the figure nobody else has for this model.

The question, stated precisely

When Clef says P(relevant) = 0.8 for a document, do roughly 80% of the documents it assigns 0.8 turn out to be judged relevant by TREC's human assessors? If yes, the score is **calibrated**, and a single relevance threshold transfers across topics — you can say “show everything above 0.7” instead of “show the top 10”. No score-based re-ranker supports that, because a BM25 score of 18.4 or a cosine of 0.62 means something different on every query.

Procedure

1. Load `clef_raw.jsonl` (5,000 decisions) and the official qrels.
2. Join on (qid, docid). **Handle unjudged documents explicitly:** TREC pooling means many retrieved documents were never assessed. Report the judged fraction, and run the analysis twice — once treating unjudged as not-relevant, once excluding them. If the two tell different stories, that itself is a finding worth a paragraph.
3. Binarise the ground truth for the main diagram: TREC grades 1 and 2 count as relevant, 0 as not relevant. Record this choice; a 2-only variant is a reasonable robustness check.
4. Bin the predicted probabilities into 10 equal-width bins over [0, 1].
5. For each bin, compute the mean predicted probability and the observed fraction of relevant documents. Plot observed against predicted, with the diagonal as the perfect-calibration reference. This is the **reliability diagram**.
6. Compute **expected calibration error** (ECE): the bin-count-weighted mean absolute gap between predicted and observed. Report Brier score alongside it.
7. Repeat for clef-flash and, if feasible, derive a comparable probability from the cross-encoder (a sigmoid over its logit) so the diagram has a baseline curve.
8. Finally, the practical test: find the single threshold that maximises F1 across all topics, then report how that fixed threshold performs per topic. Low variance across topics is the evidence that calibration is useful rather than merely present.

How to read the outcome — all three are publishable

Outcome	What it means	How we report it
Well calibrated (ECE low, points near the diagonal)	The probabilities are usable as probabilities, not just as an ordering.	Headline finding. Demonstrate threshold transfer across topics as the practical consequence.
Systematically over-confident (curve below the diagonal)	A common LLM failure. The *ordering* can still be good even when the absolute values are inflated.	Report ranking quality and calibration separately, and note that a monotone recalibration (Platt scaling or isotonic regression) would fix the values without changing the ranking.
Poorly calibrated and poor ranking	The model does not transfer to IR.	A clean negative result with a concrete diagnosis. Show which topics fail and why. This still answers RQ3.

Deliverable: `results/calibration.png` (reliability diagram with a per-bin count histogram beneath it), plus a small table of ECE, Brier score and judged fraction per system.

15 Single-GPU operations playbook

Running the whole project on one 32 GB card

Memory budget

Stage	Resident model	Approx. VRAM	Headroom on 32 GB
1 — Index	Contriever	~1 GB	Ample
2 — Generate	Qwen3-8B	~5–6 GB	Ample
3 — Retrieve	Contriever	~1 GB	Ample
4a — Cross-encoder	MiniLM-L6	~0.1 GB	Ample
4b/4c — Clef	clef (27B)	~18–20 GB	Comfortable alone; tight alongside anything else
4d — clef-flash	clef-flash	smaller	Ample
5, 6 — Evaluate, calibrate	none	0	CPU only

You are not actually VRAM-constrained — but stage anyway
Clef at ~20 GB plus Qwen3-8B at ~6 GB would in fact fit in 32 GB. But the staged design is correct independently of memory: it freezes the candidate set so the re-ranker comparison is unconfounded, it makes any stage re-runnable alone, and it means a crash costs one stage rather than the run. **We stage because it is better science, and the GPU constraint is a convenient second reason.**

Controlling residency

- Ollama loads models on demand and unloads idle ones automatically.
- `keep_alive` controls how long a model stays resident after a request. During a batch of 5,000 calls, leave it at the default (or set `"5m"`) so the model is not reloaded between calls — reloading per call would dominate the runtime.
- Set `keep_alive: 0` on the **final** call of a stage, so the GPU is clear before the next stage starts.
- Verify between stages with `nvidia-smi` and `ollama ps`. Never start a stage with a stale model resident.

Recommended execution order for a clean full run

```
python scripts/01_download.py
python scripts/02_index.py          # Contriever   -> cache + S1_bm25.trec
python scripts/07_evaluate.py --only S1     # ACCEPTANCE: nDCG@10 ~ 0.656  <-- gate
python scripts/03_generate.py       # Qwen3       -> hyde_docs.jsonl
python scripts/04_retrieve.py       # Contriever  -> S2, S3, frozen candidates
python scripts/05_rerank_ce.py      # MiniLM      -> S4
python scripts/06_rerank_clef.py --model clef       --qtype noul   # S5
python scripts/06_rerank_clef.py --model clef       --qtype score  # S6
python scripts/06_rerank_clef.py --model clef-flash --qtype noul   # S7
python scripts/07_evaluate.py       # all systems -> metrics.csv, significance.txt
python scripts/08_calibration.py    # -> calibration.png
```

Total GPU time for a full clean run is dominated by Stage 1 (corpus encoding, 20–40 min) and Stage 4 (5,000 decisions per system; well under an hour each at the published latency). Everything after Stage 1 is re-runnable without re-encoding.

16 Risks, decision log, and open questions

Living sections — update these as you go

Risk register

Risk	Mitigation	Residual
Clef's usable context is shorter than the state we send, silently truncating documents	Measure the real limit in Stage 0 and record it. Enforce `max_state_chars` and set `truncated: true` in the raw log whenever it bites.	Medium
Decision models have no CLI or SDK support yet	We call the HTTP endpoint directly, which the model card documents. Proven working in Stage 0, on day one, not in week 5.	Low
docid / embedding row misalignment	Written in one loop, in one script. The Stage 1 BM25 acceptance check is designed to catch it.	Low
Clef loses to the cross-encoder	Expected under our stated hypotheses and fully acceptable. The contribution is the measurement and the calibration study.	Low
Prompt or generation drift between runs	Prompt is versioned and frozen; generations are append-only with seeds recorded.	Low
Many unjudged documents in the pool	Report the judged fraction; run the calibration analysis both with and without unjudged documents.	Medium
Corpus re-encoding wastes days	Encode once in week 2, cache to disk, never repeat.	Low
Scope creep across four RQs	RQ1 and RQ2 are mandatory. Cut S7 first, then S6.	Medium

Decision log

Append a row whenever anything in this document changes. This is what makes the final report writable in week 8 without archaeology.

Date	Decision	Reason	By
2026-10-16	Use TREC-COVID, not Touché 2020	Graded 0/1/2 judgments are needed to test Clef's `score` type; also avoids a name clash with the Clef model	
2026-10-16	Clef used as a pointwise re-ranker, not an LLM judge	Its calibrated probability output is the right type for a ranking score	
2026-10-20	Staged file-on-disk pipeline rather than one in-process loop	Freezes the candidate set for a controlled comparison; also fits a single GPU	

Open questions to resolve early

#	Question	Resolve by
Q1	What is Clef's actual usable context length — 64K or 256K? The model card contradicts itself.	Stage 0, empirically

#	Question	Resolve by
Q2	Does the `score` question type accept a custom 3-level legend mapped to the TREC scale?	Stage 0 / early Stage 4
Q3	Do we include the topic *narrative* field in the Clef state, or only query + question?	Week 5 — test both on 5 topics, then freeze
Q4	Is a GPU build of FAISS worth installing, or is flat CPU search fast enough at 171k vectors?	Stage 1 — measure, do not guess
Q5	Should unjudged documents be excluded from calibration, or treated as non-relevant?	Report both (Section 14)

17 Timeline, ownership, and working conventions

How we work, and who holds what

Eight-week plan

Weeks	Phase	Deliverables	Gate to pass
1–2	Foundation	Environment on every machine; TREC-COVID loaded; corpus encoded and cached; BM25 baseline produced; Clef endpoint verified	**S1 nDCG@10 ≈ 0.656** and a working curl against /v1/systemone
3–4	Stage A	Prompt frozen; 400 generations written; S2 and S3 run files; top-100 candidates frozen	**S3 > S2 on nDCG@10**, and five generations read by a human
5–6	Stage B	S4, S5, S6, S7 run files; clef_raw.jsonl complete; full metrics table; significance report	All seven systems in metrics.csv with p-values
7–8	Analysis and writing	Calibration diagram and ECE; per-query win/loss; demo app; report; cleaned repo	A reader can reproduce the table from the repo alone

The week-4 checkpoint is the project's insurance
By the end of week 4 a complete Stage A results table exists, answering RQ1. Stage B depends on an unproven model; if Clef disappoints or the tooling fights us, Stage A still stands as a coherent project on its own. **Do not let Stage A slip in order to start Stage B early.**

Ownership

Roles, not names — fill in the owner column at the first team meeting. One owner per area; the owner is the person who answers questions about it in the viva.

Area	Scope	Owner
Data and indexing	Stages 1 and 3; the docid mapping; the BM25 baseline	
Generation	Stage 2; prompt design and versioning; generator ablation	
Re-ranking	Stage 4; the Clef client; checkpointing and retries	
Evaluation and analysis	Stages 5 and 6; significance; calibration; win/loss	
Document and report	This handbook; the decision log; the final write-up	

Working conventions

- **No magic numbers in scripts.** Every tunable lives in `config.yaml`. If a value appears in two places, the two places will eventually disagree and one of them will be wrong in the report.
- **Notebooks never produce a reported number.** Explore in a notebook if you like; then move the logic into a numbered script and re-run it there. Anything in `results/` must be reproducible by running a script.
- **Artifacts are append-only.** To regenerate, rename the old file with a version suffix and log why. Never overwrite generations or raw decisions.
- **Commit run files and results; do not commit caches.** `corpus_emb.npy` and `faiss.index` are regenerable and large — gitignore them. Run files are small and are the evidence.

- **One commit per stage completion**, with the acceptance-check number in the message: *“Stage 1 complete — S1 nDCG@10 = 0.6xx”*.
- **Record every number you see, even bad ones.** A failed configuration is a result; “we tried X and it scored Y” is a sentence in the report, whereas a half-remembered failure is nothing.

18 Viva preparation

The questions you will be asked, and the answers

Every team member should be able to answer all of these. The owner of each area should be able to go two levels deeper on theirs.

Question	Answer
What is the project, in one sentence?	One model imagines the document we are looking for so we can search with it; a second model reads each candidate and returns a calibrated probability that it is relevant.
What is actually new here?	Two things. Clef has never been evaluated on retrieval — its published benchmarks are intent classification and tool calling — so our numbers are the first. And we test whether its probabilities are *calibrated* against TREC's graded judgments, which re-ranking papers essentially never do.
Isn't pointwise LLM re-ranking already solved by monoT5?	Yes, and we say so. monoT5 reads the softmax over `true`/`false` tokens from a generative model — the probability is a by-product. Clef was built to emit calibrated probabilities as its primary interface. Whether that difference matters empirically is our question, not our assumption.
What if the LLM hallucinates the hypothetical document?	Three defences. The encoder is lossy — it keeps the topic and register and discards most invented specifics. We average eight generations, which cancels idiosyncratic fabrication. And Stage B re-reads the *real* document, so a hallucination that survives into retrieval still has to pass a relevance judgment.
Why not just use a cross-encoder?	We do — it is system S4, our baseline. The point is to measure whether a general decision model, which never saw ranking supervision, is competitive with a model trained on MS MARCO relevance data.
What happens if Clef loses?	That is hypothesis H3 and we expect it is possible. A supervised re-ranker beating an unsupervised one is the boring outcome. The contribution is the measurement plus the calibration finding, so a negative result is still a complete project with a concrete diagnosis.
Why TREC-COVID rather than another collection?	Graded 0/1/2 judgments, which we need to test the `score` question type and to validate calibration; 50 topics; one-line load through BEIR; and published reference numbers (BM25 = 0.656, BM25+CE = 0.757) that let us prove our pipeline is correct before we draw any conclusion from it.
Why is your Recall@100 so low?	TREC-COVID averages 493.5 relevant documents per topic. Retrieving 100 cannot recover 493, so Recall@100 is bounded low by construction. We report it as a ceiling on what the re-ranker could achieve, not as a quality score.
How do you know your BM25 implementation is right?	It reproduces the published BEIR figure of 0.656 nDCG@10. That check is a gate: we did not generate a single hypothetical document before it passed, because it validates the loader, the docid mapping and the evaluation call simultaneously.
Why should I believe a difference of 0.02 nDCG@10?	You should not, unless it is significant. Every comparison we report carries a two-sided paired t-test over the 50 topics with the p-value stated.
How does this run on one GPU?	The pipeline is staged, with files on disk between stages, so only one model is ever resident. That was partly a hardware constraint, but it is also what freezes the candidate set so that S4 through S7 are a controlled comparison.
What would you do with more time?	Three things: fuse the Clef probability with the BM25 score via reciprocal rank fusion; test recalibration (Platt or isotonic) to see whether ordering and calibration can be improved independently; and extend to a second collection to test whether any finding transfers.

19 Glossary

Shared vocabulary — nobody should have to guess

Term	Meaning
BEIR	A benchmark bundling many IR test collections behind one loader, with qrels included. Our access route to TREC-COVID.
Bi-encoder	Encodes query and document separately into vectors; fast, because documents can be indexed in advance. Contriever is one.
Calibration	Whether a predicted probability matches the observed frequency. If documents scored 0.8 are relevant ~80% of the time, the model is calibrated.
Candidate set	The top-k documents from the first stage that are handed to a re-ranker. Ours is frozen at 100 per topic.
Cross-encoder	Reads query and document *together* and outputs one relevance score. Accurate but slow, so it is used only for re-ranking.
Decision model	A model whose output is a typed, calibrated probability rather than free text. Clef is one.
Dense retrieval	Retrieval by vector similarity in a learned embedding space, as opposed to lexical term matching.
ECE	Expected calibration error — the bin-count-weighted mean gap between predicted probability and observed frequency.
FAISS	A library for storing and searching dense vectors. We use flat inner-product (exact) search.
HyDE	Hypothetical Document Embeddings. Generate a passage that would answer the query, embed it, and search with that.
Hole / unjudged	A retrieved document that no human assessor ever judged, because TREC pooling only judges what the original participating systems returned.
nDCG@10	Normalised discounted cumulative gain at rank 10. Rewards placing highly relevant documents near the top; uses graded judgments.
noul	Clef's yes/no question type, returning the probability a statement is true. Our primary ranking signal.
Pointwise re-ranking	Scoring each (query, document) pair independently, then sorting. Contrast with listwise, which orders a whole set at once.
qrels	Query relevance judgments — the human-assessed ground truth, as (qid, docid, grade) triples.
Reliability diagram	Predicted probability on the x-axis against observed relevance frequency on the y-axis. The diagonal is perfect calibration.
Run file	A ranked result list in the standard six-column TREC format. The unit of evaluation.
Zero-shot	No training on the target collection's relevance data. Our whole pipeline is zero-shot.

20 References

Papers, repositories, and models

Papers

Citation	Relevance	Link
Gao, Ma, Lin & Callan (2023). *Precise Zero-Shot Dense Retrieval without Relevance Labels.* ACL 2023.	Our anchor paper — the HyDE method	arxiv.org/abs/2212.10496
Thakur, Reimers, Rücklé, Srivastava & Gurevych (2021). *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of IR Models.* NeurIPS D&B.	Dataset access and the reference numbers we reproduce	arxiv.org/abs/2104.08663
Voorhees et al. (2020). *TREC-COVID: Constructing a Pandemic Information Retrieval Test Collection.* SIGIR Forum.	The test collection itself	arxiv.org/abs/2005.04474
Nogueira, Jiang, Pradeep & Lin (2020). *Document Ranking with a Pretrained Sequence-to-Sequence Model* (monoT5).	Closest prior art to Stage B	arxiv.org/abs/2003.06713
Sachan et al. (2022). *Improving Passage Retrieval with Zero-Shot Question Generation* (UPR).	Probability-based re-ranking, same family	arxiv.org/abs/2204.07496
Sun et al. (2023). *Is ChatGPT Good at Search? LLMs as Re-Ranking Agents* (RankGPT).	The listwise alternative we rejected	arxiv.org/abs/2304.09542
Nogueira & Cho (2019). *Passage Re-ranking with BERT.*	The cross-encoder baseline lineage	arxiv.org/abs/1901.04085
Izacard et al. (2022). *Unsupervised Dense Information Retrieval with Contrastive Learning* (Contriever). TMLR.	Our dense encoder	arxiv.org/abs/2112.09118
Thomas, Spielman, Craswell & Mitra (2024). *Large Language Models can Accurately Predict Searcher Preferences.* SIGIR 2024.	Related work on LLM relevance judgment	arxiv.org/abs/2309.10621

Repositories and models

Resource	Link	Licence / note
HyDE reference implementation	github.com/texttron/hyde	The paper's own code; see Section 8 for what we take and what we do not
BEIR	github.com/beir-cellar/beir	Apache 2.0
ranx	github.com/AmenRa/ranx	Evaluation, significance testing, fusion
sentence-transformers	sbert.net	Encoders and CrossEncoder
FAISS	github.com/facebookresearch/faiss	MIT
ir_datasets	ir-datasets.com	Cross-check for qrels parsing
Contriever	huggingface.co/facebook/contriever	Dense encoder
Cross-encoder baseline	huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2	Supervised re-ranker (S4)

Resource	Link	Licence / note
Qwen3	github.com/QwenLM/Qwen3 · ollama.com/library/qwen3	Apache 2.0 — our generator
Clef	ollama.com/library/clef	Apache 2.0 — our decision re-ranker; needs Ollama 0.35.1+
Ollama	ollama.com	Local model serving

End of handbook, version 1.0. Keep the decision log (Section 16) current — it is what makes the final report writable in week 8. When this document changes materially, bump the version on the cover and tell the team what moved.